

Designing a System That Monetizes Data

Peeps Monetization Intelligence — turning a #1 social game's first-party production data into the one decision a studio with no data team can't make: where the next dollar comes from, and which players to action for it.

Big Data Systems · Spring 2026 · National Taiwan University

葉又銘 (Yu-Ming Yeh) · R14944035

GitHub: <https://github.com/YouMingYeh/r14944035-bds-final> · Live deploy: not deployed;
dashboard runs from the repo

Thesis. Peeps is a real social game (App Store #1, **173,336** players, ~30k DAU) that monetizes at just **1.92%** paid conversion. Its production database holds something no external vendor can obtain — every player's behaviour joined to their revenue. This system turns that first-party data into a **monetization diagnostic**: it finds *why* players pay, scores *who* will, and **ranks the levers by dollar upside**. Sold to small indie social-game studios that have the same data but no data team; **Peeps is the design partner and first case study**.

173,336

PLAYERS
(PRODUCTION)

1.92%

PAID
CONVERSION

\$31.9k

LIFETIME
REVENUE ✓RC

3.9×

STATED-VS-
REVEALED

3.1×

MODEL HOLDOUT
LIFT

1. Target Customer

The customer is the **solo founder or 2–5-person studio shipping a free-to-play social or casual mobile game** — the segment that lives on in-app purchases but has **no data analyst and no data engineer**. Concretely: 10k–500k MAU, one or two engineers doing everything, monetization set by guesswork.

The job they cannot do today

They sit on the exact data Peeps has — a behavioural database plus a RevenueCat/App Store revenue feed — but cannot answer the two questions that decide their revenue: "*why do my payers pay, and who else will?*" and "*should I chase more payers, or more spend per payer?*" Their status quo is a generic analytics dashboard (RevenueCat, GameAnalytics, Firebase) that reports *what* happened and never recommends *what to do*. A senior data hire to close that gap costs ~US\$120k/yr — unjustifiable to set a \$4.99 price.

Why this is the right wedge

- **Acute pain, no in-house fix** — too small to hire a data team, too revenue-dependent to guess.
- **They already own the data** — we supply the missing intelligence layer, not the data, so there is no acquisition cost or privacy transfer.
- **Output is dollars** — the system ends in a ranked \$-upside list, so willingness to pay is trivial to anchor (§4).

2. Evidence of Demand & Willingness to Pay

The most important section, so it documents the full data-acquisition + analysis process — how we moved from "Peeps probably under-monetizes" to a quantified, per-player, revenue-validated claim.

2.1 Data acquisition — two joined first-party sources

Source	How acquired	Scale	Role
Production Postgres	Full 47.5 GB pg_dump of the live Peeps DB, restored locally; analytical tables only.	173,336 players	Behaviour, demographics, stated WTP (surveys)
RevenueCat	Pulled via API; the DB also mirrors it, so the two reconcile.	\$30.7k / 157k MAU	Revealed revenue — source of truth

The two join at `player_id` (= RevenueCat `app_user_id`). Pipeline revenue **\$31,861** reconciles with RevenueCat's independent **\$30,664** — the same first-party truth from two systems.

2.2 Why players pay — a wagering loop, not cosmetics

The dominant pcoin sink is a **competitive wagering economy**: players bet pcoins in head-to-head minigames (SPEED_CHALLENGE, GOMOKU, LOTTERY, FIGHTING) — tens of millions of pcoins in pots. Cosmetics are effectively dead (only **0.3%** of players own any). The causal signature is stark:

86.3% of payers had wagered pcoins *before* their first purchase, and **56.4%** buy on day 0. Players buy currency to keep betting — so the conversion moment is the pcoin balance running low mid-game, not a cosmetics storefront.

2.3 Stated vs revealed willingness to pay (per player)

Joining the in-app survey (stated) to real purchases (revealed) at the player level resolves a question the survey alone cannot:

Signal	Payer rate (low)	Payer rate (high)	Lift
Wagered pcoins (>0)	0.61%	2.89%	4.7×
Stated intent to pay (survey)	2.33%	9.18%	3.9×
Active ≥ 14 days	0.93%	2.97%	3.2×
Hosted ≥ 10 minigames	1.40%	4.47%	3.2×
Invited ≥ 1 friend	1.60%	3.80%	2.4×

Stated intent *is* predictive (3.9× ranking lift) but absolute conversion is tiny — most who say "yes" never pay — so pricing must be set off revealed behaviour, and **observed behaviour (wagering, 4.7×) outranks self-report**. This is the core methodological result: discount words, trust behaviour.

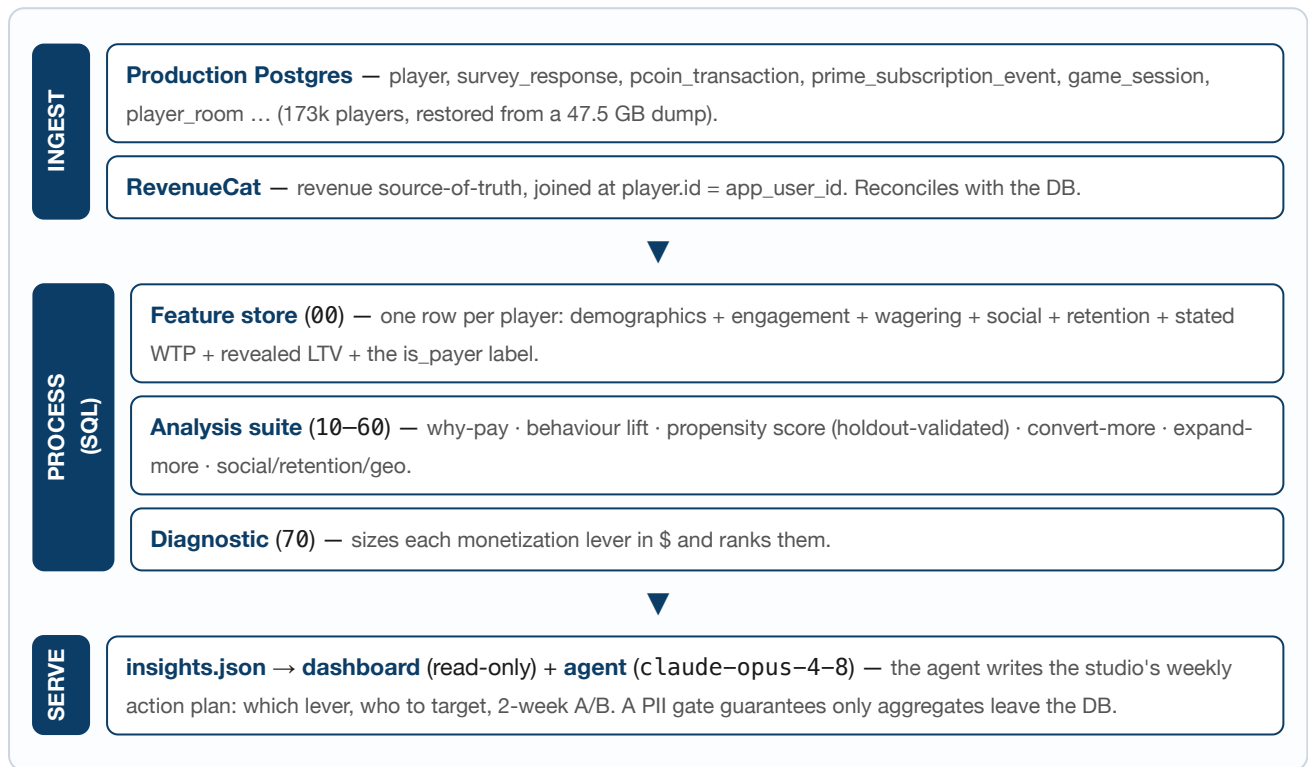
2.4 Competitor pricing & the customer's willingness to pay

Incumbents are descriptive, not prescriptive: RevenueCat / GameAnalytics / Firebase / Amplitude report metrics; none output "which lever, which players, how many dollars". Pricing-research tools (Conjoint.ly, Qualtrics) are survey platforms decoupled from the studio's revenue. For a single app this

system surfaces ~\$17k/yr of ranked, addressable upside (\$3.4); priced at \$99–\$299/month it costs a small fraction of the revenue it recovers — a clear willingness-to-pay anchor.

3. Technical System

A **SQL-first pipeline** on the production database: ingest two first-party sources, process them into a per-player feature store and an analysis suite, and serve a diagnostic to a dashboard and an LLM agent. Heavy joins (1M+ sessions, 5.8M feedings) run in Postgres; Python only orchestrates and serializes.



3.1 Data sources & ingestion

Two first-party streams: the production Postgres (behaviour + surveys) and RevenueCat (revenue). In this case study both are batched (a restored dump + an API pull); in production each is a change-data-capture / event stream into an object store, so the feature store rebuilds incrementally and refunds/late surveys reprocess idempotently.

3.2 Storage & processing — why SQL fits

The binding constraint is **joining many large behavioural tables per player** (1.0M game sessions, 5.8M pet feedings, 213k room memberships), not exotic compute. A set-based SQL feature store collapses each source per player before joining, so the whole pipeline runs in seconds and is trivially portable to a distributed warehouse (BigQuery / Spark SQL) by repartitioning on player_id — no logic change. The propensity model is a **transparent additive score** (weights on wagering, streak, activity, virality, stated intent) so a non-technical studio can audit why a player scored high; it is **validated on a 20% holdout** it was not tuned on.

3.3 The analyses (all four monetization questions)

Question	Real finding
----------	--------------

Why pay?	Wagering loop drives it; 86.3% wager before paying; pcoin_5 + pcoin_25 = \$24.5k of \$31.9k; cosmetics 0.3%.
Who pays? (behaviour diff + model)	Propensity holdout: decile-10 payer rate 6.17% vs 1.99% base = 3.1× lift , monotonic across deciles.
Convert more (breadth)	24,669 high-propensity non-payers (TW 16.5k, HK 5.3k) who already wager but never bought.
Expand more (depth)	55% buy once; whales to \$578 (p99 \$75); 3,211 pcoin buyers are not PRIME subscribers.
Social / retention / geo	Inviters pay 3.8% vs 1.6%; retention 0d→30d lifts pay 0.18%→3.16%; TW+HK = 92% of revenue.

3.4 Delivery — the diagnostic (headline output)

The system's one-line answer to "convert more vs spend more": it sizes every lever on real pools and ranks them.

Lever	Pool	Assumption	Est. annual upside
Cross-sell PRIME to pcoin buyers	3,211	5% subscribe × \$48/yr	\$7,706
Convert high-propensity non-payers	24,669	3% convert × avg payer LTV	\$7,098
Turn one-time buyers into repeat	1,786	15% 2nd purchase × incremental LTV	\$2,473

Directional upside on real pools, to be confirmed by live A/B. For Peeps the answer is "both, but PRIME first" — the subscription is barely launched (~150 real subscribers among 3.3k payers).

3.5 Scalability & cost (10× / 100×)

Per studio the marginal cost is cents (nightly SQL + a few LLM calls). At 10–100 studios the feature store becomes a multi-tenant warehouse partitioned by studio; the per-player set-based logic is unchanged. Value scales with each studio's own data, so there is no shared-data cold-start across tenants.

4. Go-to-Market Difficulties (bonus)

Obstacle	Honest assessment
Trust & adoption	The system never asserts a change — it proposes an A/B and reports the revealed result, so the studio's own data decides.
Wagering & regulation	Monetization rides a pcoin wagering loop; promoting it invites responsible-design / gambling-adjacent scrutiny (minors, odds disclosure, store policy). The system must flag, not amplify, risky targeting.
Data privacy (PDPA/GDPR)	We process inside the studio's own tenancy on pseudonymous IDs; a PII gate keeps player_id/email out of every output. The studio stays the data controller.
Cold-start	Needs the studio's own behaviour+revenue; a new game has little. Onboard with a lightweight survey SDK; revealed data comes from their existing RevenueCat.
Competition & moat	Generic analytics could add a recommendation layer; defensibility is the workflow (owning the join + the diagnostic for studios with no data team), not the (deliberately simple) model.

Unit economics	~\$17k/yr ranked upside for one mid-size app; at \$199/mo the product costs ~14% of recovered revenue with near-zero marginal cost — high gross margin once onboarding is automated.
-----------------------	--

5. Conclusion

Raw data had no value here; 173k rows read naïvely would have chased cosmetics and trusted a survey that overstates intent. Value appeared only after the data was refined — two first-party sources joined per player, the wagering driver isolated, stated intent discounted to a revealed 1.92% base rate, a holdout-validated score built, and the result handed to an agent that proposes a test rather than a dogma. That refined output — a ranked, dollar-sized list of which lever to pull and which players to action — is something a studio with no data team would genuinely pay for. The system is modest by design; the judgment it encodes is the product.

Reproducibility

Restore the dump into local Postgres, then `python3 src/run_pipeline.py` (builds the feature store + runs all SQL → `outputs/insights.json`), `python3 agent/recommend.py`, and open `dashboard/index.html`. Analytics run in SQL; the agent uses the Anthropic SDK with a deterministic offline fallback. A PII gate enforces aggregates-only; the raw production dump never leaves the gitignored `data/_private/`. README documents provenance, ethics, and limitations.